

CAMPUS EATS

Assignment 3 — Integrate an External SOAP Partner

Team ID	10
Team Leader	Annu Kumari — 20251651024
Team Member	Prerna — 20251651072
Team Member	Akanksha Upadhaya — 20251651011

1. Context (Task 1)

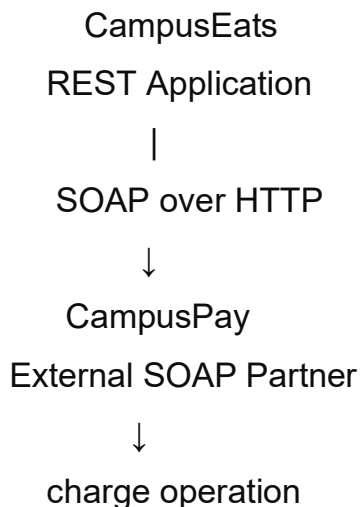
External Partner: CampusPay

Consumer: CampusEats

Operation: charge

CampusEats is primarily a REST-based application, but the payment gateway boundary is integrated using SOAP. SOAP is suitable for this external partner because it provides a strong and explicit service contract through WSDL and a structured XML message format. The SOAP interface also provides defined request, response and fault messages for the payment operation. The rest of the CampusEats application can continue using REST while the SOAP integration is kept at the external partner boundary.

The operation integrated in this assignment is charge, which is used by CampusEats to request a payment from the external CampusPay service.



Integration Flow:

CampusEats → CampusPay SOAP Service → charge operation



SOAP Response / Fault



CampusEats

2. HTTP Binding (Task 4)

CampusPay exposes the charge operation using SOAP over HTTP.

CampusEats sends the SOAP request using an HTTP POST request. The SOAP XML message is carried inside the HTTP request body. The SOAPAction value identifies the operation being requested.

HTTP Request

```
POST /soap/payment HTTP/1.1
```

```
Host: api.campuspay.example.com
```

```
Content-Type: text/xml
```

```
SOAPAction:
```

```
"http://campuseats.example.com/campuspay/charge"
```

```
Endpoint:
```

```
https://api.campuspay.example.com/soap/payment
```

1. CampusEats creates the SOAP request.



2. SOAP request is placed inside an HTTP POST.



3. HTTP request is sent to CampusPay.



4. CampusPay processes the charge operation.



5. CampusPay returns a SOAP response or SOAP Fault.

```

<soap:body>
  <pay:chargeRequest>
    <pay:orderId>ORD1001</pay:orderId>
    <pay:amount>500.00</pay:amount>
    <pay:currency>INR</pay:currency>
    <pay:cardToken>tok_demo_12345</pay:cardToken>
  </pay:chargeRequest>

```

```

===== SOAP RESPONSE =====
<?xml version="1.0" encoding="UTF-8"?>

<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:pay="http://campuseats.example.com/campuspay">

  <soap:Body>

    <pay:chargeResponse>

      <pay:paymentId>PAY2001</pay:paymentId>

      <pay:status>SUCCESS</pay:status>

      <pay:message>Payment approved</pay:message>

    </pay:chargeResponse>

  </soap:Body>

</soap:Envelope>

=====
PS C:\Users\Dell\OneDrive\Desktop\CampusEats\Assignment-3>

```

3. Discovery (Task 5)

CampusEats needs to know where the CampusPay service is located and where its WSDL contract can be obtained. The service can be discovered using a modern catalogue or registry entry. CampusEats can obtain the WSDL from the published WSDL URL and then use the contract to construct the SOAP request. A live UDDI server is not required.

Field	Registry Entry	
-------	----------------	--

Business	CampusPay
Service	CampusPay SOAP Payment Service
Operation	charge
Endpoint	https://api.campuspay.example.com/soap/payment
WSDL	https://api.campuspay.example.com/soap/payment?wsdl
tModel-style pointer	CampusPay-Charge-SOAP-Contract

Discovery Flow

Catalogue / Registry

↓

CampusPay Service Entry

↓

WSDL Contract

↓

CampusEats reads the WSDL

↓

CampusEats creates the SOAP request

4. Fault Mapping (Task 6)

CampusPay may return a partner-specific fault when a payment is declined. The partner uses the error code `card_declined`. CampusEats must not expose this partner-specific vocabulary in its own application contract.

Therefore, the integration layer maps the CampusPay error `card_declined` to the CampusEats application-level error `payment declined`.

Partner Fault	CampusEats Contract Error
<code>card_declined</code>	<code>payment declined</code>

CampusPay



card_declined



Integration / Fault Mapping



payment declined



CampusEats

When CampusEats receives the CampusPay SOAP Fault, the integration layer checks the partner error code. If the error code is card_declined, it is translated into the CampusEats error payment declined. This keeps the CampusPay-specific error vocabulary inside the integration boundary.

Example partner SOAP Fault:

```
<soap:Fault>
<faultcode>soap:Client</faultcode>
<faultstring>Payment could not be
completed</faultstring>
<detail>
<pay:errorCode>card_declined</pay:errorCode>
<pay:message>Card was declined by the payment
gateway</pay:message>
</detail>
</soap:Fault>
```

CampusEats result:

payment declined

```
pay:chargeRequest>
  <pay:orderId>ORD1001</pay:orderId>
  <pay:amount>1500.00</pay:amount>
  <pay:currency>INR</pay:currency>
  <pay:cardToken>tok_demo_12345</pay:cardToken>
</pay:chargeRequest>
```

```
===== SOAP FAULT =====
<?xml version="1.0" encoding="UTF-8"?>

<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:pay="http://campuseats.example.com/campuspay">

  <soap:Body>

    <soap:Fault>

      <faultcode>soap:Client</faultcode>

      <faultstring>
        Payment could not be completed
      </faultstring>

      <detail>

        <pay:errorCode>
          card_declined
        </pay:errorCode>

        <pay:message>
          Card was declined by the payment gateway
        </pay:message>

      </detail>

    </soap:Fault>

  </soap:Body>

</soap:Envelope>

=====
```

End-to-End Integration Flow

Step 1: CampusEats discovers the CampusPay service.



Step 2: CampusEats obtains the CampusPay WSDL.

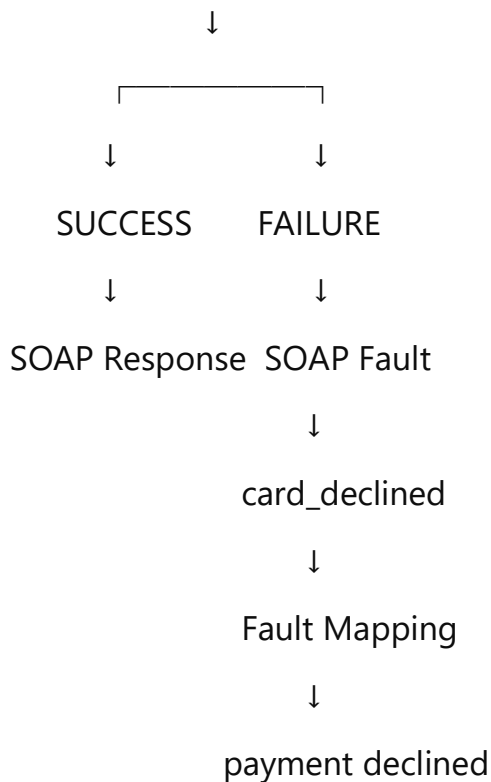


Step 3: CampusEats reads the SOAP contract.

↓
Step 4: CampusEats creates the charge SOAP request.

↓
Step 5: SOAP request is sent using HTTP POST.

↓
Step 6: CampusPay processes the charge operation.



Conclusion

The CampusEats payment boundary has been designed as an external SOAP integration with CampusPay. The integration uses the CampusPay WSDL as the service contract, SOAP over HTTP for message exchange, a catalogue or registry entry for service discovery, and fault mapping to translate the partner-specific card_declined error into the CampusEats-level payment declined error.

